

一、整合概述

zlib 是一个通用的数据压缩库，提供 deflate/inflate 压缩解压算法，被大量第三方组件（libpng、openssl、curl 等）所依赖。本文档记录了 zlib 在 OpenHarmony hispark_hi3519dv500_linux 产品中的适配过程。

版本信息：

- zlib 版本：1.2.12
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux
- 目标架构：ARM64 (aarch64)
- 编译日期：2026-05-15

二、整合步骤

1. 差异分析

通过对比参考配置（已适配 hi3519dv500）与当前 SDK 仓库 `third_party/zlib`，发现 BUILD.gn、bundle.json、CMakeLists.txt、patch 文件等所有构建配置完全一致（字节级相同），**唯一的差异**位于 minizip 扩展模块的一个源文件。

2. 源码适配修改

文件：`third_party/zlib/contrib/minizip/zip.c` 第 1488-1492 行

修改前：

```
if(uTotalOutBefore > zi->ci.stream.total_out)
{
    int bBreak = 0;
    bBreak++;
}
```

修改后：

```
// 看着像是无效代码，为通过编译，注释掉
// if(uTotalOutBefore > zi->ci.stream.total_out)
// {
//     int bBreak = 0;
//     bBreak++;
// }
```

修改原因：`deflate()` 调用后 `total_out` 只可能增加或不变，因此 `uTotalOutBefore > zi->ci.stream.total_out` 条件在正常逻辑下永远为 false，该代码块为无效死代码。但由于 BUILD.gn 开启了 `-werror`，hi3519dv500 的交叉编译工具链（`aarch64-v01c01-linux-musl-gcc`）对未使用变量 `bBreak` 触发 `-Wunused-but-set-variable` 警告并升级为编译错误。注释掉即可通过编译，且不影响任何功能。

三、问题解决记录

问题 1：未使用变量导致编译失败

错误信息：

```
contrib/minizip/zip.c:1490:17: error: variable 'bBreak' set but not used [-werror=unused-but-set-variable]
```

原因：contrib/minizip/zip.c 中存在一段无效代码，变量 bBreak 赋值后从未被使用。hi3519dv500 交叉编译工具链对 -wunused-but-set-variable 更严格，且 BUILD.gn 配置了 -werror，warning 被升级为 error。

解决方案：注释掉该无效代码块。

四、编译验证

1. 静态库编译

```
# 单独编译 zlib 静态库
hb build -T zlib
```

编译结果：

```
[OHOS INFO] [31/35] gcc cross compiler obj/third_party/zlib/contrib/minizip/libz.zip.o
[OHOS INFO] [35/35] AR libs/libz.a
[OHOS INFO] build success
[OHOS INFO] Cost time: 0:00:50
```

2. 动态库编译

```
# 单独编译 zlib 动态库
hb build -T shared_zlib
```

编译结果：

```
[OHOS INFO] [18/18] STAMP obj/third_party/zlib/shared_libz.inputdeps.stamp
[OHOS INFO] build success
[OHOS INFO] Cost time: 0:00:22
```

3. 生成文件清单

基础路径：out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux

产物	路径	大小	说明
静态库	libs/libz.a	196K	25 个源文件编译归档
动态库 (stripped)	libshared_libz.so	126K	stripped, 可部署版本

产物	路径	大小	说明
动态库 (unstripped)	unstripped/usr/lib/libshared_libz.so	141K	含调试符号

文件信息：

```
# 静态库
$ ls -lh libs/libz.a
-rw-rw-r-- 1 chenzhy223 chenzhy223 196K 5月 15 13:10 libs/libz.a

# 动态库 (stripped)
$ ls -lh libshared_libz.so
-rwxrwxr-x 1 chenzhy223 chenzhy223 126K 5月 15 13:18 libshared_libz.so

$ file libshared_libz.so
libshared_libz.so: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically
linked, stripped
```

构建配置：

- 目标平台：hispark_hi3519dv500
- 工具链：aarch64-v01c01-linux-musl-gcc
- 静态库 target：ohos_static_library("libz")
- 动态库 target：ohos_shared_library("shared_libz")
- 源文件：25 个（含 minizip 模块 ioapi.c、unzip.c、zip.c）

五、功能说明

zlib 提供以下核心能力：

压缩/解压

- deflate / inflate：基于 DEFLATE 算法的压缩解压
- compress / uncompress：内存到内存的一次性压缩解压
- gzread / gzwrite：gzip 格式文件读写

minizip 扩展

- zip.c / unzip.c：ZIP 格式文件的创建与解压
- ioapi.c：抽象 I/O 接口层

校验

- adler32：Adler-32 校验和
- crc32：CRC-32 校验和

六、使用方法

1. 在 GN 构建中依赖 zlib

```
# 依赖静态库
deps = [ "//third_party/zlib:libz" ]

# 依赖动态库
deps = [ "//third_party/zlib:shared_libz" ]
```

2. C 代码中使用

```
#include <zlib.h>

// 压缩数据
uLongf destLen = compressBound(sourceLen);
compress(dest, &destLen, source, sourceLen);

// 解压数据
uLongf destLen = expectedLen;
uncompress(dest, &destLen, source, sourceLen);
```

3. minizip 使用

```
#include "zip.h"
#include "unzip.h"

// 创建 ZIP 文件
zipFile zf = zipOpen("archive.zip", APPEND_STATUS_CREATE);

// 解压 ZIP 文件
unzFile uf = unzOpen("archive.zip");
```

七、与其他组件的关系

被依赖关系

- **openssl**: zlib 为 openssl 提供压缩支持
- **libpng**: PNG 格式内部使用 deflate 压缩, 依赖 zlib
- **curl**: 支持 gzip/deflate 内容编码, 依赖 zlib

构建配置

```
组件名: zlib
子系统: thirdparty
segment destPath: third_party/zlib
adapted_system_type: ["mini", "standard"]
```

八、注意事项

1. 仅一处源码适配

与参考配置对比确认，zlib 的适配**仅涉及一处源码注释修改**，不涉及 BUILD.gn、bundle.json 等构建配置的变更。

2. -Werror 的影响

BUILD.gn 中开启了 `-werror`（将所有警告视为错误），对交叉编译工具链的警告敏感度要求较高。后续如升级 zlib 版本，需注意 hi3519dv500 工具链可能对其他潜在警告同样严格。

3. minizip 模块

minizip（`contrib/minizip/`）是 zlib 源码树中附带的 ZIP 文件操作扩展，本次适配的改动就位于该模块。minizip 与 zlib 核心库一同编译为静态库 `libz.a`。

4. 动态库版本

BUILD.gn 中同时定义了 `ohos_shared_library("shared_libz")`，如需编译动态库版本：

```
hb build -T shared_libz
```

九、开发板验证

1. 编译测试程序

测试程序源码位于 `test_zlib.c`，直接从 zlib 源码 + 测试程序一次性交叉编译（不依赖 hb 构建系统产物）：

```
PROJ=.
ZLIB=$PROJ/third_party/zlib
CC=/opt/linux/x86-arm/aarch64-v01c01-linux-musl-gcc/bin/aarch64-v01c01-linux-musl-gcc
OUT=out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/test

mkdir -p $OUT

$CC -static \
    -I$ZLIB \
    -DHAVE_UNISTD_H \
    -wno-incompatible-pointer-types \
    -wno-strict-prototypes \
    -o $OUT/test_zlib \
    $ZLIB/adler32.c \
    $ZLIB/compress.c \
    $ZLIB/crc32.c \
    $ZLIB/deflate.c \
    $ZLIB/gzclose.c \
    $ZLIB/gzlib.c \
    $ZLIB/gzread.c \
    $ZLIB/gzwrite.c \
    $ZLIB/inffast.c \
    $ZLIB/inffast.c \
```

```
$ZLIB/inflate.c \  
$ZLIB/inftrees.c \  
$ZLIB/trees.c \  
$ZLIB/uncompr.c \  
$ZLIB/zutil.c \  
$ZLIB/contrib/minizip/ioapi.c \  
$ZLIB/contrib/minizip/unzip.c \  
$ZLIB/contrib/minizip/zip.c \  
$PROJ/test_zlib.c
```

产物信息：

```
$ file test_zlib  
test_zlib: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, with  
debug_info, not stripped  
  
$ ls -lh test_zlib  
-rwxrwxr-x 1 chenzhy223 chenzhy223 345K test_zlib
```

静态链接编译，zlib 源码与测试程序一起编译链接，不依赖 hb 构建系统产物，拷贝到板即可运行。

2. 推送到开发板

```
# 通过 adb 推送（USB 连接）  
adb push test_zlib /userdata/  
  
# 或通过串口 + tftp/nfs 传输  
# tftp -g -r test_zlib <PC_IP>
```

3. 在开发板上执行

```
chmod +x /userdata/test_zlib  
/userdata/test_zlib
```

4. 预期输出

```
=====  
zlib 移植验证 - hi3519dv500  
=====
```

[TEST] zlibversion()	PASS
-> 版本: 1.2.12	
[TEST] adler32()	PASS
-> adler32 = 0x????????	
[TEST] crc32()	PASS
-> crc32 = 0x????????	
[TEST] compress() + uncompress()	PASS
-> 原始: ?? bytes, 压缩后: ?? bytes, 压缩率: ??%	
[TEST] deflate() + inflate() 流式	PASS
[TEST] gzwrite() + gzRead() 文件读写	PASS

=====

结果：6 通过，0 失败

=====

5. 测试覆盖说明

测试项	验证内容
zlibVersion()	库可加载、符号可解析
adler32()	Adler-32 校验和计算
crc32()	CRC-32 校验和计算
compress + uncompress	一次性压缩/解压，验证数据完整性
deflate + inflate	流式压缩/解压（DEFLATE 算法核心路径）
gzWrite + gzRead	gzip 文件格式读写

判定标准：6 项全部 PASS = 移植成功。

十、参考资源

- zlib 官网：<https://zlib.net/>
- zlib 源码：<https://github.com/madler/zlib>
- OpenHarmony 文档：<https://gitee.com/openharmony>
- 参考仓库：`third_party/zlib`（已适配 hi3519dv500）